

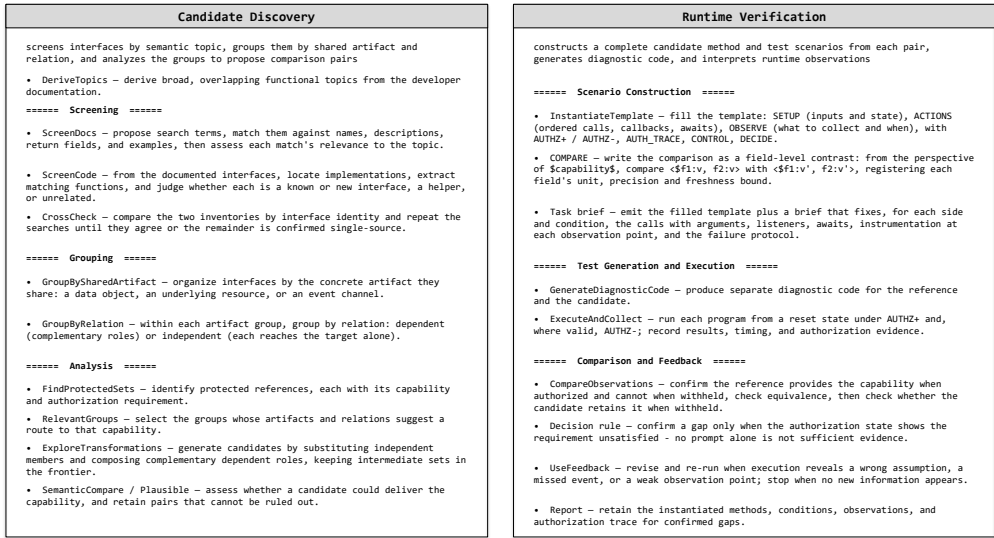


Fig. 4. The skill for permission-gap detection: Candidate Discovery and Runtime Verification.

Supplementary Material

A Skill for Permission-Gap Detection

Figure 4 summarizes the procedure as two stages, each driven by an LLM over developer documentation and framework code. Candidate Discovery screens interfaces by semantic topic, groups them by the artifact they share and by how they relate through that artifact, and analyzes the groups to propose comparison pairs, each linking a protected access method to interfaces that may form an alternative for the same capability. Runtime Verification then takes each pair, constructs a scenario that separates the two access methods and fixes the evidence required for a decision, generates diagnostic code for both sides, executes them under matched conditions that differ only in the authorization requirement under test, and interprets the observations to decide whether the candidate delivers the same capability while that requirement remains unsatisfied. Both stages share one invariant: every semantic judgement – relevance, grouping, plausibility, the comparison criterion, and the final verdict – is made by the LLM against material retrieved from documentation or code, and never from name similarity alone.

B Prompt for Permission-Gap Detection

Figure 5 reproduces the six prompts that drive the two stages. Each states the role the LLM is asked to play, the material it is given, and the form of the output it must return; curly braces mark what is supplied at run time. Three prompts belong to Candidate Discovery, (a)–(c), and three to Runtime Verification, (d)–(f). They differ in role because the judgements differ in kind: the first three decide what enters the analysis and which comparisons are worth enumerating, while the last three decide how a comparison is turned into a measurement and how the resulting observation is adjudicated. Two conventions hold throughout: a prompt never asks the LLM to re-derive a decision already made upstream, and each closes by fixing its output form, so that one step's result is consumed by the next without reinterpretation.

Prompts for Candidate Discovery (a) prompts the LLM as a domain classifier: given a candidate interface together with the documentation and code excerpts matched by the search terms, it assesses whether the interface is relevant to the topic – including interfaces that contribute only

Candidate Discovery		
(a) Interface screening – domain classifier You are a mini-program API domain classifier. Decide whether {interface}, matched by {matched_terms} in {doc_excerpt} and {code_excerpt}, belongs to domain {domain}. Assess whether the interface is relevant to the topic from the arguments and return fields it carries, including interfaces that contribute only by changing shared state or delivering an event result. Never reject for low value or no returned data; include when uncertain. Output: confirmed candidate rejected, plus one sentence of evidence.	(b) Homologous grouping – grouping analyzer You are a mini-program API grouping analyzer. Group {interface_inventory} by what they share and how they relate. Name the artifact – a data object, an underlying resource, or an event channel – and the relation: dependent (write-read, trigger-listen, open-read) for composition, or independent for substitution. The artifact must be the shared entity, not a code location or a bridging function. An interface may join several groups; differing relations split a group. Annotate each member with {permVector}, capability, and read-write role. Output per group: {id, members, sharedArtifact, category, relation}.	(c) Comparison-pair analysis – equivalence screener You are an equivalence screener for mini-program access methods. For reference {I,q} (capability {c}, requirement {p}) and {relevant_groups}, screen the equivalence hypothesis. Substitute among independent members, and join complementary roles in a dependent relation; keep intermediate sets in the frontier even if they do not yet yield {c}. Judge plausibility only – can {I,p} deliver {c} at comparable precision and freshness? – retaining all not ruled out, and reasoning from the shared artifact's mechanism and documented return fields rather than name similarity. Output: {I,q, I_p, a} with mechanism, evidence, criterion, scenario hints.
Runtime Verification		
(d) Scenario construction – scenario constructor You are a verification-scenario constructor. Fill the template for pair {pair}. For each side fix SETUP (inputs and state, parameters from {param_sensitivity}), ACTIONS (skeleton order; for a dependent relation ordered by read-write roles; for an event channel with the callback registered before the trigger and then awaited), and OBSERVE (what to collect, when delivered). Your key output is COMPARE: from the perspective of {\$(capability)\$}, compare {\$(f1:v, f2:v)} with {\$(f1:v, f2:v)}. Inferring each {\$(f)} from return fields and code and registering its unit, precision bound, and freshness bound – these fix what execution must measure. Let AUTHZ+ satisfy all requirements and AUTHZ- withhold only {pi}, holding every other requirement and operating condition fixed. Output: the filled template plus a task brief for code generation.	(e) Code generation – diagnostic code generator You are a diagnostic code generator for mini-program permission tests. Translate {scenario} and {task_brief} into code without re-deriving the logic. Reset state before each side; keep the call order, register callbacks before the trigger, and record every timeout as “not triggered”. Log fields named exactly as {fieldRegistry}, collecting trigger and callback sides separately for event channels. Grant everything to the protected side and nothing extra to the candidate, and end with three-branch assertions. Any timeout, missing callback, or error must emit an inconclusive record, never silence. Output: scenario registration code and the test plan JSON.	(f) Result judgement – verdict adjudicator You are a verdict adjudicator for runtime verification results. Adjudicate {vf} against the COMPARE expression. Judge first whether the run succeeded and the observations are trustworthy. Then normalize units, read both sides from {observations}, and mark each field equivalent, not equivalent, or undecidable against its bound. A gap holds only if the candidate still delivers with {pi} withheld and the auth trace confirms {pi} was unchecked – “no popup” alone is no evidence. Rule it out when both sides are blocked or diverge, and return inconclusive whenever data is missing: missing data never means “vulnerability absent”. Finally route: let a confirmed gap enter the output, trigger another trial when the outcome is inconclusive, or return to candidate discovery, scenario construction, or code generation stating the concrete new information – or stop if there is none. Output: verdict, field-level evidence, insufficiency, and routing.

Fig. 5. The six prompts: (a)–(c) for candidate discovery, (d)–(f) for runtime verification.

by changing shared state or delivering an event result – and it errs toward inclusion. (b) prompts it as a grouping analyzer: it names the concrete artifact the interfaces share – a data object, an underlying resource, or an event channel – and the relation through which they reach it, dependent or independent, since a differing relation splits a group and one interface may join several. (c) prompts it as an equivalence screener: for a protected reference with its capability and authorization requirement, it generates candidates by substituting independent members and by joining complementary roles in a dependent relation, and judges plausibility only – whether the candidate could deliver the capability at comparable precision and freshness – retaining everything that cannot be ruled out. The three therefore separate membership, structure, and hypothesis: the first decides what enters the inventory, the second decides which comparisons are worth enumerating, and the third decides which of them survive to verification.

Prompts for Runtime Verification (d) prompts the LLM as a scenario constructor: it fills SETUP, ACTIONS, and OBSERVE for each side – resolving the required component state and inputs, the calls with their concrete arguments, and what the mini-program can observe – and writes COMPARE, turning the comparison criterion into observable checks on what the two sides deliver. It also sets the two authorization conditions: AUTHZ+ satisfies all applicable requirements for both methods, whereas AUTHZ- withholds only pi and holds the other requirements and operating conditions fixed. (e) prompts it as a diagnostic code generator that translates the filled scenario and its task brief into code without re-deriving the test logic: the code resets the state before each side, registers callbacks before the triggering call, logs fields using exactly the names registered in COMPARE, and emits an inconclusive record on any timeout, missing callback, or error. (f) prompts it as a verdict adjudicator: it first judges whether the run succeeded and the observations are trustworthy, then compares the two sides under the criterion defined in COMPARE – confirming that the reference provides the capability when its requirement is satisfied and cannot provide it when that requirement is withheld, checking equivalence, and finally checking whether the candidate retains the capability when the same requirement is withheld – and routes the result: a confirmed gap enters the output, an inconclusive outcome triggers another trial through execution feedback, and a result that contradicts an upstream judgement is sent back to candidate discovery or scenario construction with the concrete new information that motivated the return.